PROJECT PROPOSAL • SOFTWARE ENGINEERING (PPL)

ReqLint: Requirement Analyzer via NLP

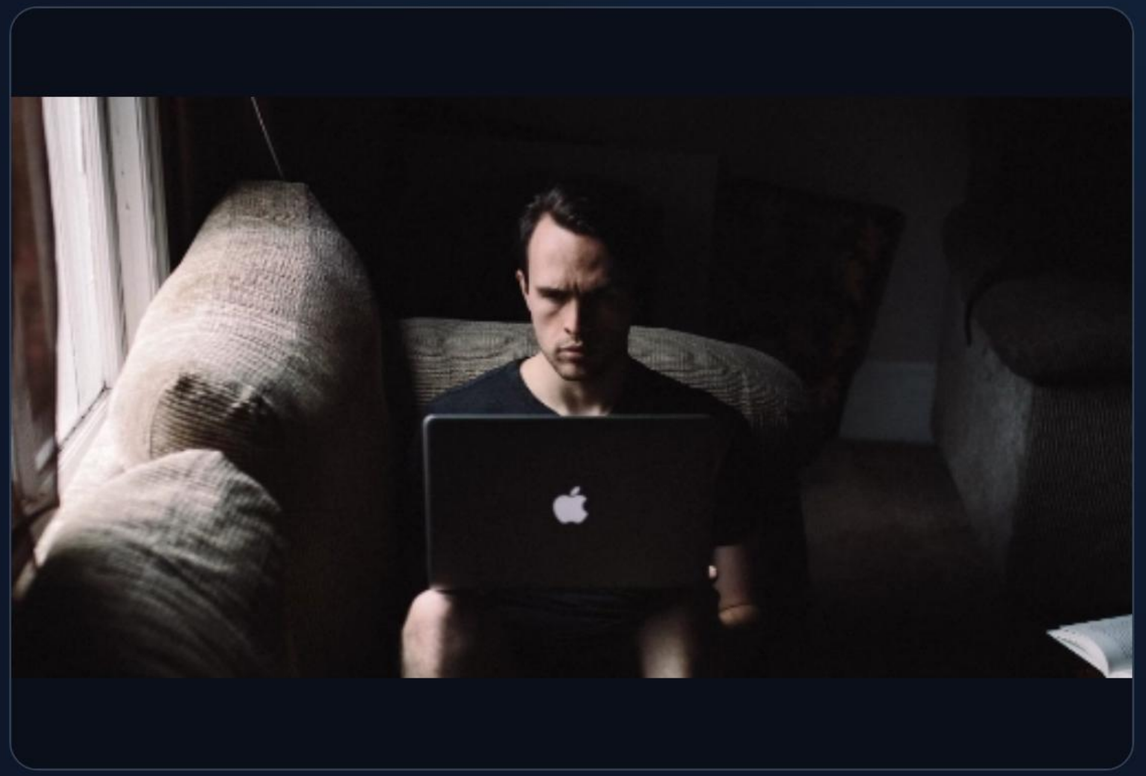
An Automated Quality Assurance & Ambiguity Detection Tool for Software Requirement Specifications (SRS) based on IEEE 29148 Standard.

The Core Problem

Why do software projects fail early in the Software Development Life Cycle (SDLC)?

Shift-Left Quality Assurance

- **Early Defect Origins:** Over 60% of software defects stem from ambiguous or incomplete initial requirements.
- **Exponential Cost Increase:** Fixing a requirement flaw in production costs up to 100× more than catching it in the design phase.
- **Subjective Language:** Words like *"fast"*, *"user-friendly"*, and *"secure"* lack measurable acceptance criteria.
- **The Solution:** Automated NLP-driven "linting" for SRS documents before sprint commitment.



Standardized Analysis Framework

ISO/IEC/IEEE 29148 Standard

Ensures user requirements adhere to international software engineering criteria: Unambiguous, Complete, Consistent, Verifiable, and Traceable.

INVEST Criteria for Agile

Evaluates User Stories to be Independent, Negotiable, Valuable, Estimable, Small, and Testable before entering sprint backlogs.

Three Pillars of Requirement Audit



Ambiguity Detection

Identifies weak directives, subjective adjectives, and passive voice that obscure system actor responsibility.



Completeness Audit

Checks structural requirements (Role-Action-Benefit) and verifies presence of testable acceptance criteria.



Actionable Revision

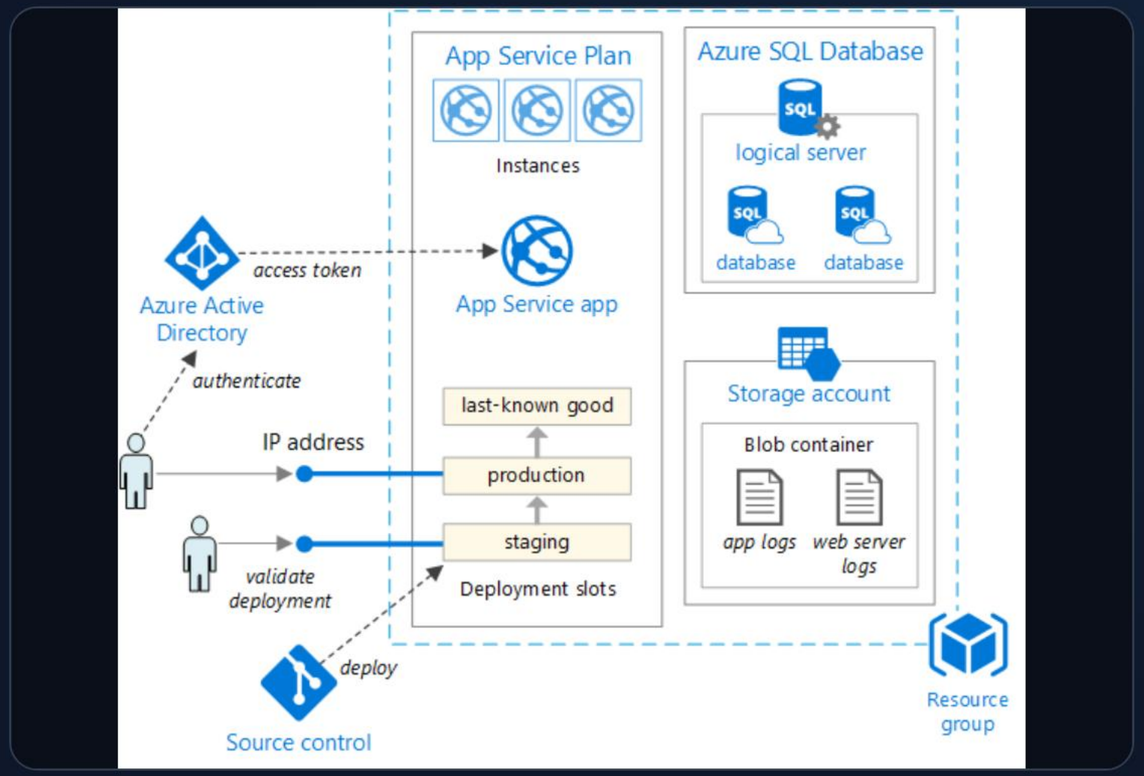
Generates real-time suggestions and concrete rewriting examples using fine-tuned NLP heuristics & SLM reasoning.

Architecture & Tech Stack

A modular, robust software architecture tailored for seamless 4-member team execution.

System Architecture Overview

- **Decoupled Micro-services:** Next.js Frontend communicates via REST API with FastAPI backend.
- **Hybrid NLP Pipeline:** Fast deterministic rule-matching (spaCy) coupled with LLaMA-3/Groq API for semantic reasoning.
- **Async Task Processing:** Redis + Celery queue handles heavy document batch analysis smoothly.
- **Persisted Audit Logs:** PostgreSQL stores requirement history and version revisions for progress tracking.



Detailed Technology Stack

LAYER	TECHNOLOGIES SELECTED	KEY PURPOSE / JUSTIFICATION
Frontend UI	Next.js 14, Tailwind CSS, Monaco Editor	Interactive text highlighting, real-time diff viewer, diff editor.
Backend API	FastAPI (Python 3.11), Pydantic v2	High-performance async I/O API with automatic Swagger docs.
AI / NLP Engine	spaCy, LangChain, Groq LLaMA-3, HuggingFace	POS tagging, rule-based "smell" linting, and context-aware suggestions.
Database & Queue	PostgreSQL, Redis, Celery	Relational data tracking, audit logs, background task processing.
DevOps & QA	Docker, PyTest, Cypress, GitHub Actions	Containerized environments, automated unit & integration testing.

Team Division & Roles (4 Members)

PM & Architect

Farhan (Lead)

System architecture design, SRS project spec, metric scoring rubrics, sprint management.

AI / NLP Engineer

Member 2

NLP parsing pipeline, POS & NER rule-matching, LLM prompt engineering & benchmark evaluation.

Backend Engineer

Member 3

FastAPI endpoints, DB schema, async Redis worker queue, auth & document history API.

Frontend + QA

Member 4

Interactive requirement editor, real-time score visualizer, system unit & integration testing.

Semester Project Roadmap

Weeks 1 – 4

SRS & Pitching, Rule Rubric definition, DB Schema design.

Weeks 5 – 8

Core NLP Pipeline, FastAPI integration, Basic Editor UI.

Weeks 9 – 12

Hybrid SLM Reasoning, Real-time Highlighting, QA Testing.

Weeks 13 – 16

System Evaluation, Final PPL Demo & Paper Submission.

Research & Publication Potential

Hybrid

Rule-Based NLP + SLM Reasoning

High Academic Value

Target Publication: IEEE Conference or Sinta 2 Journal.

Novelty Focus: Empirical comparison showing that hybrid NLP pipelines yield significantly lower false-positive rates than raw LLMs for Requirement Engineering.

Ready for Pitching

ReqLint: Empowering Software Engineers with Flawless Requirements.

Software Engineering Project (PPL) • Semester 5

Image Sources



https://media2.dev.to/cdn-cgi/image/width=800%2Cheight=%2Cfit=scale-down%2Cgravity=auto%2Cformat=auto/https%3A%2F%2Fdev-to-uploads.s3.amazonaws.com%2Fuploads%2Farticles%2Fofr5xetx373asnfeys2h.png

Source: dev.to



https://learn.microsoft.com/en-us/dotnet/architecture/modern-web-apps-azure/media/image5-5.png

Source: learn.microsoft.com